Faculty of Computer Applications

AI-Based Language Translator

Master of Computer Applications

Submitted To:

Mr. Amarjeet

Technical Trainer

Faculty of CSED

Submitted By:

Team Name: MCA2B-08

Group Leader: Md Irshad

Total Member: 12

Invertis University, Bareilly [UP]

(2025-26)

DECLARATION

I, Md Irshad, hereby declare that the project entitled "AI-Based Language Translator" submitted in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (MCA) is our original work.

This project has been carried out under the guidance and supervision of our respected faculty Mr. Amarjeet, Department of Computer Applications.

The work presented in this project has not been submitted previously, either in full or in part, for the award of any other degree or diploma to any institution or university. All the sources of information used in this project have been properly acknowledged.

This project has been completed as a collaborative effort by our team members under the team name "MCA2B-08", and each member has contributed equally towards the successful completion of the project.

Team Details:
Team Name: MCA2B-08
Class: MCA Section B

S. No.	Student Name	Contribution
1	Md Irshad (Leader)	100%
2	Mohd Arshad	100%
3	Suneel Kumar Singh	100%
4	Mohd Amaan	100%
5	Mohd Amaan	100%
6	Mohd Fazal	100%
7	Akshay Kumar	100%
8	Siddharth Singh	100%
9	Sayud Alam	100%
10	Pawan Kumar Singh	100%
11	Shahroz Khan	100%
12	Gourav Kumar	100%

Place: _____
Date: _____

Signature of Team Leader
(Md Irshad)
MCA Section B

CERTIFICATE

This is to certify that the project entitled "AI-Based Language Translator" has been successfully carried out by Md Irshad (Team Leader) along with the team members of MCA Section B (Team Name: MCA2B-08) during the academic session 2025–2026, in partial fulfillment of the requirements for the award of the degree of Master of Computer Applications (MCA).

The project work has been completed under my guidance and supervision. The team has shown sincere effort, dedication, and teamwork throughout the development of this project.

I am satisfied with the quality of work carried out by the students and recommend this project for evaluation.

(Project Guide Signature)
Mr. Amarjeet
Technical Trainer
Department of Computer Applications

(External Examiner)

Date: _____
Place: _____

ACKNOWLEDGMENT

We would like to express our heartfelt gratitude to our respected project guide **Mr. Amarjeet**, Technical Trainer, Department of Computer Applications, for his/her constant guidance, encouragement, and valuable suggestions throughout the development of this project. His continuous support and insightful feedback helped us understand the subject deeply and improve the overall quality of our work. Without his/her guidance, this project would not have been completed successfully.

We are also sincerely thankful to all the faculty members of the Department of Computer Applications for providing us with the necessary knowledge, resources, and academic support required during the course of this project. Their teachings and guidance played an important role in strengthening our technical and analytical skills.

We would like to extend our appreciation to our institution for providing a positive and motivating learning environment that encouraged us to explore new ideas and technologies. The facilities and support provided by the college helped us in completing our project efficiently.

We are equally grateful to our family members and friends for their continuous support, patience, and motivation throughout the project duration. Their encouragement helped us stay focused and dedicated towards our work.

Finally, we express our sincere thanks to all the members of **MCA Section B (Team MCA2B-08)** for their cooperation, teamwork, and collective effort. Working together on this project has been a valuable learning experience, and each member's contribution played a significant role in the successful completion of this project.

ABSTRACT

This project focuses on the development of an **AI-Based Language Translator** designed to overcome language barriers and enable effective communication between users of different languages. In today's digital world, communication across multiple languages is essential in fields such as education, business, and online platforms. Traditional translation methods are often time-consuming and may lack accuracy, creating a need for an automated and efficient solution.

The main objective of this project is to build a system that can translate text from one language to another using Artificial Intelligence and Natural Language Processing (NLP). The system accepts user input in the form of text, processes it through a translation model, and provides the output in the selected target language.

The project is implemented using Python and NLP libraries, ensuring fast and reliable translation. It supports multiple languages and delivers results in real-time. The system is designed with a simple and user-friendly interface, making it easy to use.

Although the system performs well for basic translations, it may face challenges with complex sentences and contextual meanings. Future improvements can include voice translation and enhanced AI models for better accuracy.

Keywords: Artificial Intelligence, Language Translation, Natural Language Processing (NLP), Machine Learning, Text Processing, Python

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Background

In today's digital era, communication across different languages has become a common necessity due to globalization and the rapid growth of internet-based platforms. People from different regions, cultures, and linguistic backgrounds interact frequently in areas such as education, business, travel, and social media. However, language barriers often create difficulties in understanding and effective communication.
To overcome this challenge, modern technologies like Artificial Intelligence (AI) and Natural Language Processing (NLP) have been widely adopted. These technologies enable machines to understand human language, process textual data, and generate meaningful outputs. Language translation systems are one of the most important applications of NLP, allowing users to convert text from one language to another quickly and efficiently.
Earlier, translation was done manually or through rule-based systems, which were time-consuming and less accurate. With the advancement of machine learning and transformer-based models, translation systems have become more intelligent, capable of understanding context, sentence structure, and semantics.

This project focuses on developing an AI-Based Language Translator that utilizes modern NLP techniques and transformer models to provide real-time and accurate translation, thereby reducing language barriers and improving communication.

1.2 Motivation

The primary motivation behind this project is to address the challenges caused by language barriers in everyday communication. In many real-life situations such as education, business interactions, travel, and online communication, people often face difficulties in understanding different languages. This creates a gap in communication and limits access to information.
Traditional translation methods are either manual or rely on basic tools, which are time-consuming and may not always provide accurate results. With the advancement of Artificial Intelligence and Natural Language Processing, it has become possible to automate the translation process and improve both speed and accuracy.
This project is motivated by the need to develop a smart and efficient system that can perform real-time language translation using modern AI techniques. It also provides an opportunity to gain practical knowledge of transformer-based models and web application development using Flask.

By building this system, the aim is to create a user-friendly solution that simplifies communication and demonstrates the practical application of AI in solving real-world problems.

1.3 Problem Statement

In today's interconnected world, communication across different languages is essential, yet it remains a significant challenge for many individuals. People often encounter difficulties in understanding and translating text written in foreign languages, which creates barriers in accessing information, education, and effective communication.
Existing translation methods, including manual translation and basic online tools, are either time-

consuming or may not always provide accurate and context-aware results. These systems often fail to handle complex sentence structures, idiomatic expressions, and contextual meanings properly.

Additionally, many users require a simple and efficient solution that can provide real-time translation without complexity. There is a need for a system that can process input text quickly, understand its meaning, and generate accurate translations across multiple languages.
Therefore, this project aims to develop an AI-Based Language Translator that addresses these challenges by using advanced Natural Language Processing techniques and transformer-based models to provide efficient, accurate, and real-time translation.

1.4 Objectives

The main objectives of this project are:

- To develop an AI-based system for language translation
- To provide real-time translation of text
- To support multiple languages
- To design a user-friendly interface
- To demonstrate the practical use of AI and NLP

1.5 Scope of the Project

The scope of this project includes the development of a text-based language translation system that can be used in various fields such as education, communication, and digital platforms. It can help users understand foreign languages easily and improve accessibility of information.

The system can be further enhanced by adding features such as voice translation, image-based text recognition, and integration with mobile or web applications.

1.6 Project Overview

The AI-Based Language Translator is a web-based application designed to provide real-time translation of text using Artificial Intelligence and Natural Language Processing (NLP). The system allows users to enter text and select a target language through a simple and user-friendly interface.
Once the input is provided, the system performs basic preprocessing and sends the text to a transformer-based translation model. This model analyzes the structure and meaning of the sentence and generates an accurate translation in the selected language.
The backend of the system is developed using Python and Flask, which manages user requests and connects the frontend with the translation module. The translation functionality is implemented using pre-trained MarianMT models, which support multiple languages and ensure efficient performance.

The translated output is displayed instantly on the interface, making the system fast, reliable, and suitable for real-time communication. This project demonstrates how AI and web technologies can be combined to build an effective multilingual translation system.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing System Analysis

In the current scenario, language translation is commonly performed using manual methods or online translation tools. Manual translation requires human effort, which is time-consuming and may lead to inconsistencies, especially when handling large volumes of text.

Various online tools such as Google Translate are widely used for quick translation across multiple languages. These systems provide fast results and are easily accessible. However, they often face limitations in understanding the correct context, tone, and meaning of complex sentences. As a result, the translated output may not always be accurate or meaningful.

Traditional translation systems were mainly rule-based, relying on predefined grammar rules and dictionaries. While these systems work for simple sentences, they fail to handle complex language structures and contextual variations effectively.

Although modern systems have improved using machine learning techniques, many existing solutions still depend on internet connectivity and lack full contextual understanding. Therefore, there is a need for more advanced translation systems that can provide accurate, real-time, and context-aware results using modern AI techniques.

2.2 Research Gap

After analyzing the existing systems, several gaps have been identified. Although current tools provide basic translation functionality, they lack deep contextual understanding and may produce incorrect or incomplete translations for complex inputs.

Another major limitation is the lack of personalization and adaptability in many translation systems. Most tools do not learn from user interactions or improve over time. Moreover, offline translation capabilities are limited, which restricts usability in areas with poor internet connectivity.

There is also a need for systems that are simple, efficient, and capable of providing real-time translation with better accuracy. Therefore, this project focuses on developing an AI-based language translator that attempts to address these limitations by using modern technologies such as Natural Language Processing and machine learning techniques.

2.3 Proposed System

To overcome the limitations of existing translation systems, this project proposes an **AI-Based Language Translator** that utilizes advanced Natural Language Processing (NLP) techniques and transformer-based models for accurate and real-time translation.

The proposed system is designed as a web-based application where users can enter text and select the desired target language through a simple interface. The input text is processed using a backend system developed in Python and Flask, which handles user requests and manages the translation process.

The core functionality of the system is implemented using pre-trained transformer models, specifically MarianMT models, which are capable of understanding sentence structure and context more effectively than traditional rule-based systems. This allows the system to generate more accurate and meaningful translations.

Additionally, the system includes a model caching mechanism to improve performance by reducing repeated loading time. The overall design ensures fast response, better accuracy, and support for multiple languages.
Thus, the proposed system provides a more efficient, intelligent, and user-friendly solution for multilingual communication.

2.4 Advantages of Proposed System

The proposed AI-Based Language Translator offers several advantages over traditional and existing translation systems:

- **Improved Accuracy:**
  The use of transformer-based models enables better understanding of sentence structure and context, resulting in more accurate translations.

- **Real-Time Translation:**
  The system provides quick responses, allowing users to translate text instantly without delay.

- **Multi-Language Support:**
  It supports multiple languages such as Hindi, Bengali, Urdu, French, and German, making it useful for diverse users.

- **User-Friendly Interface:**
  The web-based interface is simple and easy to use, allowing even non-technical users to perform translations easily.

- **Efficient Performance:**
  The implementation of model caching reduces loading time and improves system performance.

- **Scalability:**
  The system can be easily extended to support more languages and advanced features in the future.

- **Integration of Modern Technologies:**
  The use of Artificial Intelligence, Natural Language Processing, and web technologies makes the system more powerful and reliable compared to traditional methods.

These advantages make the proposed system more efficient, accurate, and suitable for real-world applications.

CHAPTER 3: METHODOLOGY

3.1 Data Collection

In this project, data collection is based on dynamic user input rather than a predefined dataset. The system is designed to accept text directly from users through the web interface, which can be in the form of words, sentences, or short paragraphs.
Since the application performs real-time translation, it does not rely on stored data. Instead, it processes the input provided by the user at runtime. For testing and evaluation purposes, various sample sentences in English were used to verify the functionality and accuracy of the system.
Additionally, different types of input such as simple phrases, questions, and moderately complex sentences were tested to ensure that the system performs effectively under different conditions. This approach makes the system flexible and suitable for real-world applications where input can vary significantly.
The collected input data serves as the primary source for the translation process and is directly passed to the preprocessing and translation modules.

3.2 Data Preprocessing

Data preprocessing is a crucial step in the translation process, as it ensures that the input text is clean and properly structured before being passed to the translation model. Raw input provided by users may contain unnecessary characters, inconsistent formatting, or irregular spacing, which can affect translation accuracy.

To handle this, the system performs basic preprocessing operations on the input text. These include:

- Removal of unwanted characters and symbols
- Conversion of text into a normalized format
- Handling of extra spaces and formatting issues
- Tokenization, where the input text is broken down into smaller units (tokens) for processing

Tokenization plays an important role in Natural Language Processing, as it helps the model understand the structure and sequence of words in a sentence. The tokenizer converts the input text into a format that can be processed by the transformer-based model.

These preprocessing steps improve the efficiency of the system and enhance the quality of the translated output by ensuring that the model receives clean and well-structured input data.

3.3 Translation Process

The translation process is the core component of the system, where the actual conversion of text from the source language to the target language takes place. After preprocessing, the cleaned input text is passed to a transformer-based translation model, specifically the MarianMT model.
The process begins with tokenization, where the input text is converted into numerical tokens that the model can understand. These tokens are then processed by the pre-trained model, which has been trained on large multilingual datasets. The model analyzes the grammatical structure, context, and meaning of the input sentence.

Using its learned knowledge, the model generates the translated output in the desired target language. The generated tokens are then decoded back into readable text format.

This entire process is known as model inference, where the system uses a trained model to produce output without additional training. The use of transformer-based models ensures better accuracy and context understanding compared to traditional rule-based or statistical methods.
The translation is performed in real-time, allowing users to receive quick and efficient results.

3.4 System Workflow

The overall workflow of the AI-Based Language Translator follows a structured sequence of steps to ensure smooth and efficient operation. The system integrates the user interface, preprocessing module, and translation engine to provide real-time results.

The step-by-step workflow is as follows:

1. **User Input:**
   The user enters text into the input field through the web interface and selects the desired target language.

2. **Input Processing:**
   The system receives the input and performs basic preprocessing, such as cleaning and tokenization, to prepare the text for translation.

3. **Model Processing (Translation Engine):**
   The processed text is passed to the transformer-based MarianMT model, which analyzes the sentence structure and context to generate the translated output.

4. **Output Generation:**
   The translated text is decoded into a readable format and prepared for display.

5. **Display Result:**
   The final translated output is displayed on the user interface instantly.

This workflow ensures efficient communication between different components of the system and enables fast, accurate, and real-time translation. It also highlights the integration of Artificial Intelligence with web technologies in solving language translation problems.

CHAPTER 4: SYSTEM DESIGN

4.1 Tools and Technologies Used

The development of the AI-Based Language Translator utilizes a combination of programming languages, libraries, and development tools to ensure efficient and accurate translation.

- **Python:** The core programming language used to implement the translation logic and handle backend processing. It provides simplicity and strong support for AI and NLP applications.
- **Natural Language Processing (NLP) Libraries:** Libraries such as NLTK or SpaCy are used for text processing tasks like tokenization, normalization, and language understanding.
- **Translation Libraries (e.g., googletrans / transformers):** These libraries are used to perform the actual translation by converting text from the source language to the target language using pre-trained models.
- **Visual Studio Code:** Used as the primary development environment for coding, debugging, and testing the application efficiently.
- **Web Technologies (HTML, CSS, JavaScript):** Used to design a simple, responsive, and user-friendly interface for user interaction.
- **Operating System:** The system is compatible with standard operating systems such as Windows, macOS, and Linux, ensuring flexibility in deployment.
- **Internet Connectivity:** Required for accessing online translation APIs or models, enabling real-time translation functionality.

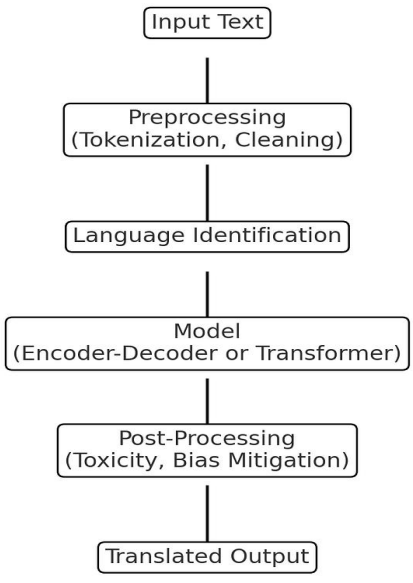
These technologies together create a robust system capable of handling multilingual input and providing fast and accurate output.

4.2 Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) illustrates how data flows through different stages of the system, from user input to the final translated output. It provides a clear understanding of the internal working and interaction between components.

Flow of the System:

User Input (Text)
↓
Input Validation & Language Selection
↓
Data Preprocessing
↓
Translation Module (AI/NLP Engine)
↓
Output Formatting
↓
Translated Output



Detailed Explanation:

1. **User Input:**
   The process begins when the user enters text into the system and selects the desired target language.

2. **Input Validation & Language Selection:**
   The system checks whether the input is valid (not empty or invalid characters) and confirms the selected source and target languages.

3. **Data Preprocessing:**
   The input text is cleaned, normalized, and tokenized to prepare it for translation.

4. **Translation Module (AI/NLP Engine):**
   The processed text is passed to the translation model, which analyzes the structure and meaning of the sentence and generates the translated output.

5. **Output Formatting:**
   The translated text is formatted properly to ensure readability and correctness.

6. **Translated Output:**
   Finally, the translated result is displayed to the user.

The DFD helps in visualizing how the system processes data step by step, ensuring clarity, efficiency, and smooth functioning of the translation process.

4.3 System Architecture

The system architecture defines the structure and interaction between different components of the translator system.

The architecture is divided into three main layers:

- **Input Layer:**

  This layer accepts input from the user in the form of text. It also allows the user to select the source and target languages.

- **Processing Layer:**

  This is the core layer where all processing takes place. It includes data preprocessing and the AI/NLP-based translation module. This layer is responsible for analyzing and converting the input text.

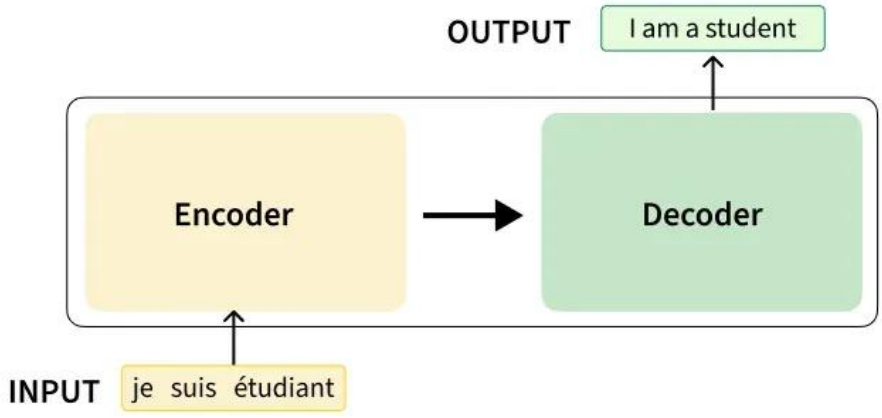
- **Output Layer:**

  This layer displays the translated text to the user in a clear and readable format.

Architecture Flow:

[User Input]
↓
[Preprocessing]
↓
[AI/NLP Translation Engine]
↓
[Output Display]

This layered architecture ensures smooth data processing, easy maintenance, and efficient system performance.



CHAPTER 5: IMPLEMENTATION

5.1 System Development

The AI-Based Language Translator system was developed using Python with a modular and scalable approach. The system is divided into two main components: the backend translation engine and the web-based user interface.

The backend is implemented using transformer-based models from the Hugging Face library, while the frontend is developed using Flask, a lightweight Python web framework. The project structure is organized into separate files such as `app.py` for routing and `translator_model.py` for translation logic.

This modular design ensures better code management, easy debugging, and future scalability.

5.2 User Interface and Routing

The user interface is developed using HTML templates integrated with Flask. The system provides multiple pages such as Home, Translator, About, and Contact.

The routing is handled in the Flask application (`app.py`), where different URLs are mapped to specific functions. The main translation functionality is implemented in the `/translate` route.

When a user submits text through the interface:

- The input text is captured using form data
- The selected target language is retrieved
- The request is processed using the backend translation function
- The translated output is displayed on the same page

This ensures smooth interaction between the user and the system.

5.3 Translation Engine Implementation

The core translation functionality is implemented in the `translator_model.py` file using transformer-based models.
The system uses pre-trained MarianMT models from Hugging Face for translating text into multiple languages such as Hindi, Bengali, Urdu, French, and German.

Key features of the implementation include:

- Dynamic model selection based on target language
- Tokenization of input text for processing
- Use of pre-trained AI models for translation
- Decoding of output into readable format

Code Snippet:

```
from transformers import MarianMTModel, MarianTokenizer

LANGUAGE_MODELS = {
   "Hindi": "Helsinki-NLP/opus-mt-en-hi"
}

def translate_text(text, language):
   model_name = LANGUAGE_MODELS[language]

   tokenizer = MarianTokenizer.from_pretrained(model_name)
   model = MarianMTModel.from_pretrained(model_name)

   tokens = tokenizer(text, return_tensors="pt", padding=True)
   translated = model.generate(**tokens)

   return tokenizer.decode(translated[0], skip_special_tokens=True)
```

This implementation demonstrates how transformer models are used to generate accurate translations.

5.4 Model Optimization and Caching

To improve performance, the system uses a caching mechanism. Instead of loading the translation model every time a request is made, the model is loaded once and stored in memory.
This reduces processing time and improves response speed significantly. The cache stores both the tokenizer and the model for each language, ensuring efficient reuse.

5.5 System Integration and Output

The frontend (Flask application) and backend (translation engine) are fully integrated to provide a seamless experience.
The workflow is as follows:

- User inputs text through the web interface
- Flask captures the request and sends it to the translation module
- The translation engine processes the text using AI models
- The translated result is returned and displayed on the webpage

The system provides real-time translation with a user-friendly interface, demonstrating the effective implementation of AI and NLP technologies in a web-based application.

CHAPTER 6: RESULTS & ANALYSIS

6.1 System Output

After successful implementation of the AI-Based Language Translator, the system is able to translate text from English into multiple target languages such as Hindi, Bengali, Urdu, French, and German.

The application provides a web-based interface where users can input text, select the desired target language, and receive the translated output instantly. The translation process is fast and efficient due to the use of transformer-based models.

The system consists of different sections:

- Input field for entering text
- Language selection dropdown
- Translate button
- Output display area

The output is generated in real-time and displayed clearly on the screen, making the system user-friendly and easy to operate.

6.2 Output Examples

Some sample results obtained from the system are:

Input Text	Target Language	Translated Output
Hello	Hindi	नमस्ते
How are you?	French	Comment ça va ?
Good Morning	German	Guten Morgen
Thank you	Urdu	شکریہ

These examples demonstrate that the system can accurately translate simple and commonly used sentences.

6.3 Performance Analysis

The system performs efficiently in translating text in real-time. The use of transformer-based models ensures better accuracy compared to traditional rule-based systems.

Performance observations:

- Fast response time after initial model loading
- High accuracy for simple and medium-length sentences
- Slight delay during the first-time model loading (due to downloading and initialization)
- Supports multiple languages effectively

6.4 Observations

The following observations were made during testing:

- The system provides accurate translations for general sentences
- The interface is simple and easy to use
- Model caching improves performance after first use
- Complex sentences may sometimes produce less accurate results
- Internet connection is required for loading models initially

6.5 Limitations

Despite its effectiveness, the system has some limitations:

- Initial model loading time is high
- Accuracy may decrease for complex or long sentences
- Requires internet connection for downloading models
- Limited to predefined languages

6.6 Summary of Results

The results show that the AI-Based Language Translator is capable of performing real-time multilingual translation effectively. The system successfully demonstrates the practical use of Artificial Intelligence and Natural Language Processing in solving communication problems.

The overall performance is satisfactory, and the system meets the objectives defined at the beginning of the project.

CHAPTER 7: CONCLUSION & FUTURE SCOPE

7.1 Conclusion

The AI-Based Language Translator developed in this project successfully demonstrates how Artificial Intelligence and Natural Language Processing (NLP) can be used to overcome language barriers. The system is capable of translating text from English into multiple languages such as Hindi, Bengali, Urdu, French, and German in real-time.

The project integrates transformer-based models with a web-based interface using Flask, providing a smooth and user-friendly experience. The system efficiently processes user input, performs translation, and displays the output instantly.

Through this project, we gained practical knowledge of AI, NLP, and web development. It also helped in understanding how modern technologies can be applied to solve real-world communication problems.

Overall, the system meets its objectives and provides satisfactory performance for basic and moderate translation tasks.

7.2 Limitations

Despite the successful implementation, the system has certain limitations:

- Initial loading time of models is high
- Accuracy may reduce for complex or long sentences
- Requires internet connection for downloading models
- Limited number of supported languages
- Does not fully understand context in some cases

7.3 Future Scope

The project can be further improved and extended in several ways:

- Integration of voice-based translation (speech-to-text and text-to-speech)
- Support for more languages and regional dialects
- Development of a mobile application version
- Implementation of offline translation models
- Use of advanced AI models for better context understanding
- Integration with chatbots or real-time communication platforms

With these enhancements, the system can become more powerful, accurate, and suitable for large-scale real-world applications.

REFERENCES

1. Hugging Face Transformers Documentation.
   Available at: https://huggingface.co/docs/transformers
2. MarianMT Models – Helsinki NLP.
   Available at: https://huggingface.co/Helsinki-NLP
3. Flask Documentation – Python Web Framework.
   Available at: https://flask.palletsprojects.com
4. Python Official Documentation.
   Available at: https://docs.python.org/3/
5. Natural Language Processing (NLP) – Overview and Concepts.
   Available at: https://www.ibm.com/topics/natural-language-processing
6. Research Papers on Machine Translation and NLP (various sources).
7. Online Tutorials and Learning Resources (used during project development).

ANNEXURES / APPENDIX

Appendix A – Source Code
Includes Python files such as app.py and translator_model.py used in the project.

Translator.py

```
from transformers import MarianMTModel, MarianTokenizer

LANGUAGE_MODELS = {
    "English": None,
    "Hindi": "Helsinki-NLP/opus-mt-en-hi",
    "Bengali": "Helsinki-NLP/opus-mt-en-bn",
    "Urdu": "Helsinki-NLP/opus-mt-en-ur",
    "French": "Helsinki-NLP/opus-mt-en-fr",
    "German": "Helsinki-NLP/opus-mt-en-de"
}

model_cache = {}

def translate_text(text, language):

    if language not in LANGUAGE_MODELS:
        return "Language not supported"

    # English selected → return same text
    if language == "English":
        return text

    # load model only when needed
    if language not in model_cache:

        model_name = LANGUAGE_MODELS[language]

        tokenizer = MarianTokenizer.from_pretrained(model_name)
        model = MarianMTModel.from_pretrained(model_name)

        model_cache[language] = (tokenizer, model)

    tokenizer, model = model_cache[language]

    tokens = tokenizer(text, return_tensors="pt", padding=True)

    translated = model.generate(**tokens)

    result = tokenizer.decode(translated[0],
skip_special_tokens=True)

    return result
```

```
def available_languages():
    return list(LANGUAGE_MODELS.keys())
```

App.py

```
from flask import Flask, render_template, request
from translator_model import translate_text, available_languages

app = Flask(__name__)
# Home Page
@app.route("/")
def home():
    return render_template("chome.html")
# Translator Page
@app.route("/translate", methods=["GET", "POST"])
def translate():
    result = ""
    if request.method == "POST":

        text = request.form["text"]
        language = request.form["language"]

        result = translate_text(text, language)

    languages = available_languages()

    return render_template(
        "translate.html",
        translated_text=result,
        languages=languages
    )
# About Page
@app.route("/about")
def about():
    return render_template("about.html")


# Contact Page
@app.route("/contact")
def contact():
    return render_template("contact.html")

@app.route('/templates/rock-paper-scissors/index.html')
def game():
    return render_template('game.html')


if __name__ == "__main__":
    app.run(debug=True, port=5001)
```

Appendix B– HTML Templates
Contains Code of the application including Home Page, Translator Page.

Home Page.html

```
<!DOCTYPE html>
<html>
<head>
<title>AI Translator</title>
<style>

body{
margin:0;
font-family:Arial;
background:linear-gradient(135deg,#1c1f2f,#2c3e50);
color:white;
overflow-x:hidden;
}

.menu-btn{
font-size:28px;
padding:20px;
cursor:pointer;
}
.sidebar{
position:fixed;
left:-250px;
top:0;
width:250px;
height:100%;
background:#0f111a;
padding-top:60px;
transition:0.4s;

}
.sidebar a{

display:block;
padding:18px;
text-decoration:none;
color:#ccc;
font-size:20px;

}

.sidebar a:hover{
background:#1f2235;
color:white;
}
.close-btn{
position:absolute;
```

```
top:10px;
right:20px;
font-size:30px;
cursor:pointer;
}
/* main content *
.main{
padding:60px;
text-align:center;
}
.main h1{
font-size:40px
}
.main p{
font-size:18px;
color:#ccc;

}
</style>
<script>
function openMenu(){
document.getElementById("sidebar").style.left="0";
}
function closeMenu(){
document.getElementById("sidebar").style.left="-250px";
}
</script>
</head>
<body>

<!-- hamburger icon -->
<div class="menu-btn" onclick="openMenu()">☰</div>

<!-- sidebar -->

<div class="sidebar" id="sidebar">
<div class="close-btn" onclick="closeMenu()">×</div>
<a href="/">Home</a>
<a href="/translate">Translation</a>
<a href="/about">About</a>
<a href="/contact">Contact</a>
</div>

<!-- main welcome section -->

<div class="main">
<h1>Welcome to the AI Language Translator</h1>
<p>Break language barriers using Artificial Intelligence</p>
</div>
</body>
</html>
```

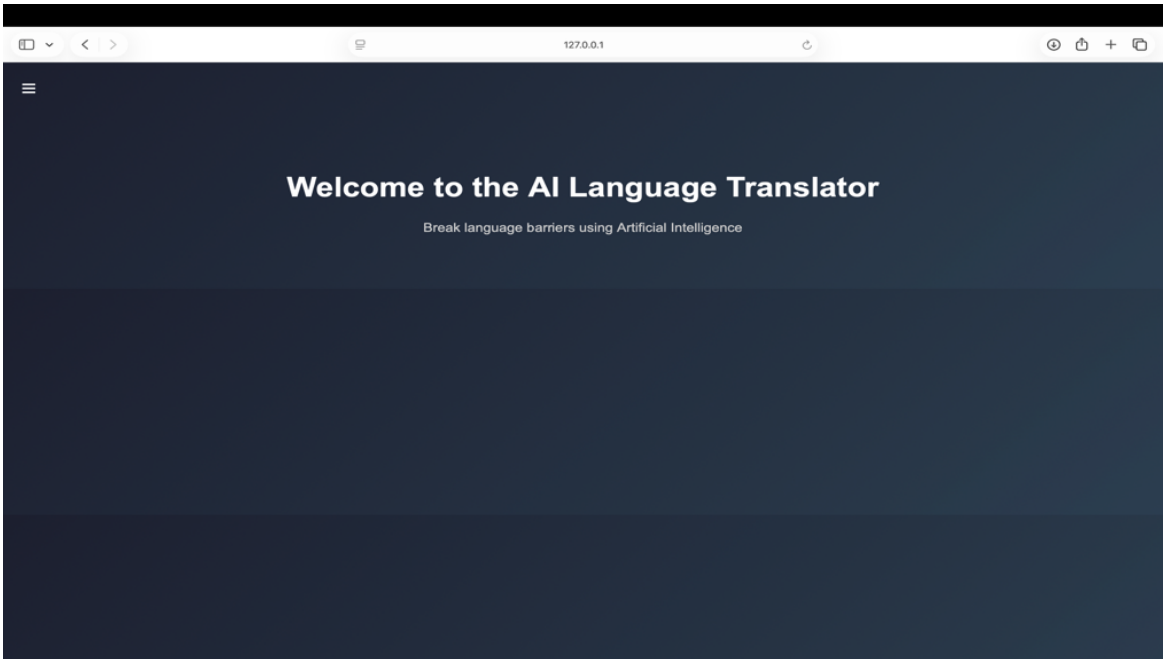
Translator.html

```
<!DOCTYPE html>
<html>
<head>
<title>Translator</title>
<style>
body{
margin:0;
font-family:Arial;
background:linear-gradient(120deg,#4facfe,#00f2fe);
}
.container{
width:650px;
margin:120px auto;
background:white;
padding:35px;
border-radius:15px;
box-shadow:0 10px 25px rgba(0,0,0,0.2);
text-align:center;
}
textarea{
width:95%;
height:120px;
padding:10px;
border-radius:8px;
border:1px solid #ccc;
}
select{
margin-top:10px;
padding:10px;
}
button{
margin-top:10px;
padding:10px 25px;
border:none;
background:#4facfe;
color:white;
border-radius:8px;
}

.result{
margin-top:20px;
background:#f2f2f2;
padding:15px;
border-radius:10px;
}

</style>
```

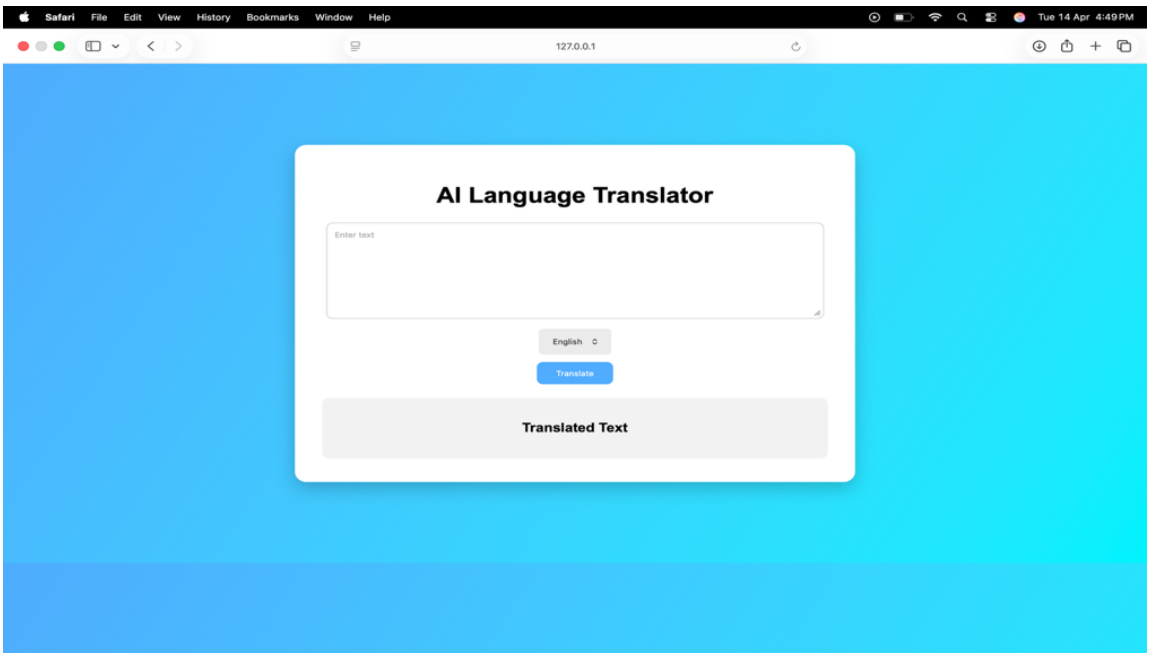
```
</head>
<body>
<div class="container">
<h1>AI Language Translator</h1>
<form method="POST">
<textarea name="text" placeholder="Enter text"></textarea>
<br>
<select name="language">
{% for lang in languages %}
<option value="{{lang}}">{{lang}}</option>
{% endfor %}

</select>
<br>
<button type="submit">Translate</button>

</form>
<div class="result"
<h3>Translated Text</h3>
<p>{{translated_text}}</p>
</div>
</div>
</body>
</html>
```

Appendix C – User Interface Screens
Contains screenshots of the application including Home Page, Translator Page, and Output.

Home Page



Translator Page



About Page